

CO2 AT3 - DEBUGGING & OPTIMIZATION TASK QUESTIONS

Name: ADHITHYA.S

Register Number: 192412352

Course Code: ITA0606

Course Name: MACHINE LEARNING

11. Debugging a Poorly Performing SVM Classifier

Symptom

Very low accuracy on BOTH training and test sets. Since even training accuracy is poor, this points to underfitting (high bias) rather than overfitting — the model is too constrained or mismatched to learn the underlying pattern at all.

Possible Causes

Cause	Explanation
Improper kernel selection	Using a linear kernel on data that is not linearly separable (or vice-versa) prevents the model from forming a decision boundary that fits the classes.
Incorrect C (regularization)	A very small C over-penalizes misclassification tolerance, forcing an overly simple/wide margin that underfits the data.
Incorrect gamma (RBF kernel)	A very small gamma makes the decision boundary too smooth/broad, so the model can't capture the true class structure; a very large gamma overfits instead.
Poor feature scaling	SVM is distance-based; unscaled features (very different ranges) distort the margin computation and dominate the decision boundary, degrading performance on all data.

Systematic Debugging Steps

- Confirm training vs test accuracy gap — both low confirms a bias (underfitting) problem, ruling out overfitting-focused fixes.
- Check preprocessing pipeline — verify StandardScaler/MinMaxScaler was applied to all numeric features before training.
- Visualize the data (2-D/PCA projection) to judge whether classes are linearly separable or need a non-linear kernel.
- Try a simple baseline (e.g., linear kernel, default parameters) and compare against RBF/polynomial kernels.
- Inspect class balance — a skewed dataset can also depress accuracy even with a correct kernel.

Optimization Techniques

- Normalize/standardize all features before training so no single feature dominates the margin calculation.
- Kernel tuning: switch to an RBF or polynomial kernel if the data is non-linearly separable.
- Grid Search (or Randomized Search) with k-fold cross-validation over C and gamma jointly, since they interact — e.g., search C = {0.1, 1, 10, 100}, gamma = {0.001, 0.01, 0.1, 1}.
- Increase C moderately if the model is under-fitting, to allow a tighter, more expressive decision boundary.
- Add or engineer more informative features if the underlying signal is genuinely weak in the current feature set.

12. Debugging Imbalanced K-Means Clusters

Symptom

Clusters produced are highly imbalanced (some very large, some tiny) and don't reflect the true structure of the data.

Possible Causes

Cause	Explanation
Incorrect choice of K	Too few clusters forces dissimilar points together; too many splits a natural group into fragments — both distort the true distribution.
Poor centroid initialization	Random initialization can place centroids close together or in an unrepresentative region, causing K-Means to converge to a poor local

	optimum with skewed cluster sizes.
Presence of outliers	K-Means uses the mean to update centroids; extreme outliers pull centroids away from the true cluster centers, distorting cluster boundaries and sizes.
Unscaled features	Features with larger numeric ranges dominate the Euclidean distance calculation, effectively clustering on one dimension only.

Debugging Approaches

- Visualize the data (PCA or t-SNE projection) to inspect the natural cluster shapes and count before/after clustering.
- Plot cluster sizes and centroid positions to spot clusters dominated or distorted by a few extreme points.
- Compute the Within-Cluster Sum of Squares (WCSS) and Silhouette Score across a range of K values to assess cluster quality.
- Re-run K-Means multiple times with different random seeds — large variation in results confirms an initialization sensitivity problem.

Improvements

- Elbow Method: plot WCSS against K and pick the K at the "elbow" point where the rate of decrease sharply flattens, to choose a more appropriate number of clusters.
- K-Means++ initialization: spreads out initial centroids intelligently (probabilistically favoring points far from existing centroids), greatly reducing the chance of poor local optima and imbalanced results.
- Outlier removal: detect and remove/cap extreme points (e.g., using IQR or z-score thresholds) before clustering, or use a robust scaler.
- Feature scaling: standardize all features so each contributes proportionately to the distance metric.
- If clusters remain irregular in shape or contain noise, consider alternative algorithms such as DBSCAN (density-based, handles outliers and non-spherical clusters) or Gaussian Mixture Models (soft, probabilistic clustering).

13. Debugging an RNN with Inconsistent Time-Series Predictions

Symptom

The RNN produces inconsistent outputs and fails to capture long-term dependencies in the sequence.

Possible Causes

Cause	Explanation
Vanishing gradients	In a plain RNN, gradients are repeatedly multiplied through many time steps during backpropagation-through-time; they shrink toward zero, so weights linked to earlier time steps barely update — the model effectively 'forgets' distant past information.
Insufficient sequence length	If training sequences are too short, the model never gets exposed to the long-range patterns it needs to learn, regardless of architecture.
Improper architecture design	A vanilla RNN cell (no gating), too small a hidden state, or no normalization layers limits the network's ability to retain and use long-term information.
Exploding gradients / no clipping	In some cases gradients can instead grow very large, causing unstable, erratic weight updates — a related but opposite issue to vanishing gradients.

Debugging Steps

- Track the magnitude of gradients per layer/time-step during training — gradients shrinking toward zero for earlier steps confirms vanishing gradients; extremely large/NaN values confirm exploding gradients.
- Plot training and validation loss curves — a loss that plateaus early and stays flat suggests the network isn't learning long-range signal.
- Test the model on artificially shortened sequences to see if performance improves, isolating whether sequence length or the architecture itself is the bottleneck.
- Check the current cell type (plain RNN vs LSTM/GRU) and hidden state size in the architecture definition.

Optimization Techniques

- Replace vanilla RNN cells with LSTM or GRU units — their gating mechanisms (forget/input/output gates) explicitly control what information is retained or discarded, allowing gradients to flow more effectively across many time steps.
- Apply gradient clipping (cap the gradient norm to a fixed threshold) to prevent exploding gradients and stabilize training.
- Normalize input sequences (e.g., per-feature z-score normalization) so the network trains on a consistent input scale, improving convergence stability.

- Consider additional techniques for very long sequences: attention mechanisms, residual/skip connections, or Layer Normalization within the recurrent layers.

14. Debugging a Poorly Performing Naïve Bayes Text Classifier

Symptom

The Naïve Bayes classifier performs poorly on real-world text data containing noisy and irrelevant features (words).

Possible Causes

Cause	Explanation
Violation of feature independence assumption	Naïve Bayes assumes all words are conditionally independent given the class, but in real text, words are correlated (phrases, co-occurring terms) — heavy violations degrade probability estimates.
Improper preprocessing	Without cleaning (stop-word removal, punctuation/number stripping, stemming/lemmatization), the vocabulary balloons with noisy, uninformative tokens that dilute the signal from genuinely predictive words.
Imbalanced dataset	Naïve Bayes uses class priors directly in its probability calculation; a skewed class distribution biases predictions toward the majority class.
Zero-frequency problem	A word never seen with a particular class in training gets a zero conditional probability, which can zero out the entire posterior for that class unless smoothing is applied.

Debugging Strategies

- Inspect the vocabulary size and a sample of top features — an unusually large vocabulary full of rare/uninformative tokens signals a preprocessing gap.
- Check the class distribution in the training data for imbalance.
- Evaluate per-class precision/recall (not just overall accuracy) to see if the model is systematically biased toward the majority class.
- Run an ablation: remove very rare or very common words and re-evaluate to quantify how much noisy features are hurting performance.

Improvements

- Feature selection: use Chi-square test, mutual information, or TF-IDF weighting to keep the most discriminative words and drop noisy/irrelevant ones.
- Apply Laplace (add-one) or Lidstone smoothing so unseen word–class combinations get a small non-zero probability instead of zeroing out predictions.
- Balance the dataset via oversampling the minority class, undersampling the majority class, or using class-weighted priors.
- Strengthen preprocessing: tokenization, stop-word removal, stemming/lemmatization, and setting min/max document-frequency thresholds to drop overly rare or overly common terms; consider bigrams to partially capture word dependencies.

15. Debugging a Reinforcement Learning Navigation Agent

Symptom

The RL agent fails to learn an optimal policy for the navigation task and shows erratic behavior during and after training.

Possible Causes

Cause	Explanation
Sparse rewards	If the agent only receives a reward at the very end (e.g., reaching the goal), it gets little feedback during most of an episode, making it hard to assign credit to the actions that actually helped — learning becomes slow or stalls.
Improper exploration–exploitation balance	Too much early exploitation causes premature convergence to a suboptimal policy; too much exploration (or exploration that never decays) prevents the agent from ever settling into a consistent, learned policy — producing erratic behavior.
Incorrect reward function design	A poorly designed reward can unintentionally incentivize the wrong behavior (reward hacking), or reward magnitudes that are too large/small can destabilize learning.
Other contributing factors	Too high a learning rate, too few training episodes, or an unstable/non-converging function approximator (e.g., a poorly tuned neural network policy) can also cause erratic behavior.

Debugging Approaches

- Plot cumulative reward per episode over training — a flat or highly noisy curve (rather than a rising trend) indicates the agent isn't consistently improving.
- Log state-action visitation frequency/heatmaps to check whether the agent is exploring the environment adequately or getting stuck in a small region.
- Manually inspect the reward values returned by the environment during sample episodes to verify they actually align with the intended goal.
- Track the epsilon (exploration rate) value over training episodes to confirm it is decaying as intended.
- Test the algorithm on a simplified version of the environment to isolate whether the issue is the RL algorithm itself or the environment/reward design.

Optimization Strategies

- Reward shaping: add intermediate, denser reward signals (e.g., a small reward for reducing distance to the goal) to guide learning instead of relying solely on a sparse terminal reward.
- Epsilon-decay tuning: gradually reduce the exploration rate (epsilon) from a high initial value to a low value over training, so the agent explores broadly early on and increasingly exploits its learned policy later.
- Use more stable policy optimization methods such as Proximal Policy Optimization (PPO) or Actor-Critic methods, which are designed to balance stable policy updates with effective exploration, instead of a vanilla Q-learning/policy-gradient approach.
- Normalize state and reward inputs, tune the learning rate, and (for value-based methods) use experience replay to break correlation between consecutive updates and stabilize training.